

Reservoir statistics analysis

This notebook documents Compulsory Assignment 1 for IND320. It reads the original `reservoirs.csv` file supplied with the course material. The source file is never modified.

Data source: `Curriculum/D2Dbook/data/reservoirs.csv`

1. Read the source data

The code first locates the IND320 project root. It then reads the CSV with Pandas into `df_raw`, which keeps the original Norwegian column names.

```
In [1]: from pathlib import Path

import pandas as pd
import matplotlib.pyplot as plt

def find_project_root(start_path: Path) -> Path:
    """Find the IND320 root by looking for the course data folder."""
    for candidate in (start_path, *start_path.parents):
        source_file = candidate / 'Curriculum' / 'D2Dbook' / 'data' /
        'reservoirs.csv'
        if source_file.exists():
            return candidate

    raise FileNotFoundError('Could not find
Curriculum/D2Dbook/data/reservoirs.csv.')

PROJECT_ROOT = find_project_root(Path.cwd().resolve())
DATA_PATH = PROJECT_ROOT / 'Curriculum' / 'D2Dbook' / 'data' / 'reservoirs.csv'

# Read the original CSV. This operation does not change the source file.
df_raw = pd.read_csv(DATA_PATH)

print(f'Read {len(df_raw):,} rows from: {DATA_PATH}')
df_raw.head()
```

Read 14,877 rows from:

`C:\Users\elias\Documents\IND320\Curriculum\D2Dbook\data\reservoirs.csv`

```
Out[1]:
```

	dato_Id	omrType	omrn timer	iso_aar	iso_uke	fyllingsgrad	kapasitet_TWh	fylling_TWh
0	1995-09-03	EL	4	1995	35	0.944721	21.079208	19.913979
1	2020-11-15	EL	1	2020	46	0.974361	6.003264	5.849344
2	2011-08-28	EL	4	2011	34	0.786499	21.079208	16.578772
3	1998-07-05	EL	3	1998	27	0.793969	8.920932	7.082947
4	2011-03-27	EL	4	2011	12	0.300820	21.079208	6.341054

2. Create an English analysis version

`df` is a copy used for analysis. Renaming its columns does not alter `df_raw` or the source CSV. The `date` column is converted to a date type, and the rows are sorted from oldest to newest before later plots are made.

```
In [2]: COLUMN_NAMES = {
    'dato_Id': 'date',
    'omrType': 'area_type',
    'omrn timer': 'area_number',
    'iso_aar': 'iso_year',
    'iso_uke': 'iso_week',
    'fyllingsgrad': 'fill_level_ratio',
    'kapasitet_TWh': 'capacity_twh',
    'fylling_TWh': 'stored_energy_twh',
    'neste_Publiseringdato': 'next_publication_date',
    'fyllingsgrad_forrige_uke': 'previous_week_fill_level_ratio',
    'endring_fyllingsgrad': 'weekly_fill_level_change',
}

# Make an analysis copy, rename the headers, and prepare the main time
variable.
df = df_raw.rename(columns=COLUMN_NAMES).copy()
df['date'] = pd.to_datetime(df['date'])
df = df.sort_values(['date', 'area_type',
    'area_number']).reset_index(drop=True)

df.head()
```

Out[2]:

	date	area_type	area_number	iso_year	iso_week	fill_level_ratio	capacity_twh	stoi
0	1995-01-08	EL	1	1995	1	0.621467	6.003264	
1	1995-01-08	EL	2	1995	1	0.630947	34.043590	
2	1995-01-08	EL	3	1995	1	0.602376	8.920932	
3	1995-01-08	EL	4	1995	1	0.558864	21.079208	
4	1995-01-08	EL	5	1995	1	0.616448	17.390587	

3. Initial inspection

A relevant first view shows the first rows after sorting, the dimensions of the data set, the date range, and the data types.

In [3]:

```
print(f'Data set size: {df.shape[0]:,} rows and {df.shape[1]} columns')
print(f'Date range: {df["date"].min().date()} to {df["date"].max().date()}')

display(df.head())
display(df.dtypes.rename('data_type').to_frame())
```

Data set size: 14,877 rows and 11 columns

Date range: 1995-01-08 to 2026-09-06

	date	area_type	area_number	iso_year	iso_week	fill_level_ratio	capacity_twh	store
0	1995-01-08	EL	1	1995	1	0.621467	6.003264	
1	1995-01-08	EL	2	1995	1	0.630947	34.043590	
2	1995-01-08	EL	3	1995	1	0.602376	8.920932	
3	1995-01-08	EL	4	1995	1	0.558864	21.079208	
4	1995-01-08	EL	5	1995	1	0.616448	17.390587	

	data_type
date	datetime64[us]
area_type	str
area_number	int64
iso_year	int64
iso_week	int64
fill_level_ratio	float64
capacity_twh	float64
stored_energy_twh	float64
next_publication_date	str
previous_week_fill_level_ratio	float64
weekly_fill_level_change	float64

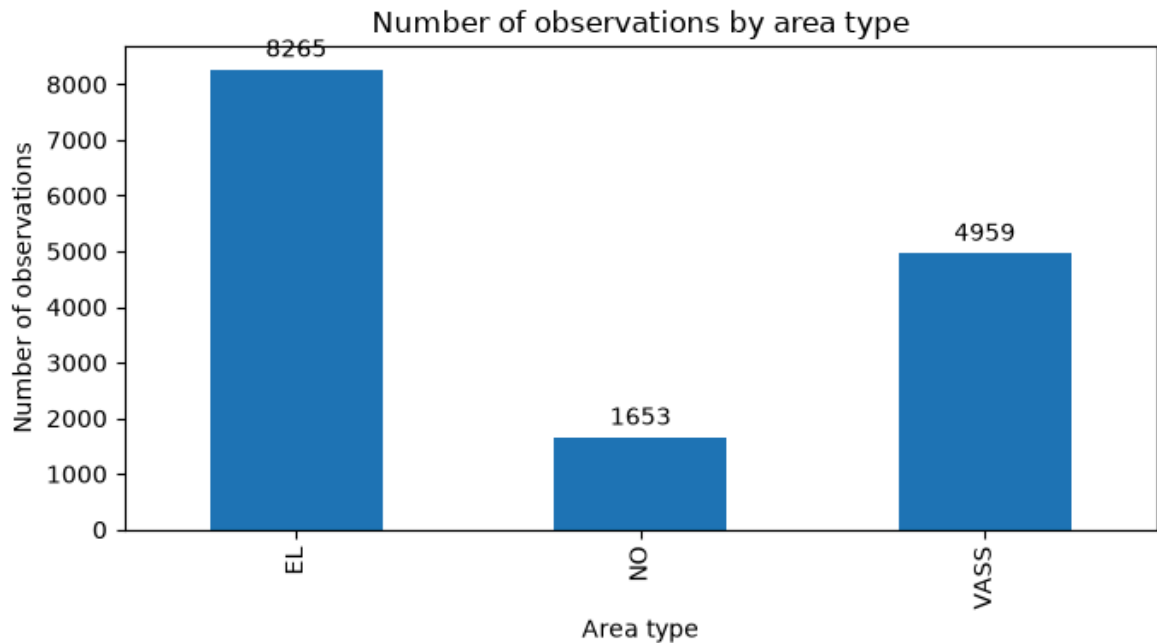
4. Distribution of observations by area type

This bar chart counts the number of rows in each `area_type` category. It describes the structure of the data set, not the reservoir level or capacity of an area.

```
In [4]: # Count how many observations belong to each area type.
area_type_counts = df['area_type'].value_counts().sort_index()

ax = area_type_counts.plot(
    kind='bar',
    color='#1f77b4',
    figsize=(7, 4),
)
ax.set_title('Number of observations by area type')
ax.set_xlabel('Area type')
ax.set_ylabel('Number of observations')
ax.bar_label(ax.containers[0], padding=3)

plt.tight_layout()
plt.show()
```



5. Separate plots for measurement columns

The two area systems are alternative ways of grouping the same reservoirs. Each time-series figure therefore compares electricity price areas (EL) on the left with watercourse areas (VASS) on the right. The national series (NO-0) is included in both panels as a shared reference. Capacity is shown as bars rather than a time series because it is a size measure for each area.

```
In [5]: # Create readable labels for the area series and select the national reference.
plot_data = df.assign(
    area_label=df['area_type'] + '-' + df['area_number'].astype(str)
)
national_data = plot_data.loc[
    (plot_data['area_type'] == 'NO') & (plot_data['area_number'] == 0)
].copy()

# Store settings for the time-series measurements. Capacity is handled
separately.
LINE_PLOT_SETTINGS = {
    'fill_level_ratio': {
        'title': 'Reservoir fill level by area system',
        'y_label': 'Fill level (%)',
        'multiplier': 100,
    },
    'stored_energy_twh': {
        'title': 'Stored energy by area system',
        'y_label': 'Stored energy (TWh)',
        'multiplier': 1,
    },
    'previous_week_fill_level_ratio': {
        'title': 'Previous-week reservoir fill level by area system',
        'y_label': 'Previous-week fill level (%)',
        'multiplier': 100,
    },
    'weekly_fill_level_change': {
        'title': 'Weekly change in reservoir fill level by area system',
```

```

        'y_label': 'Weekly change (percentage points)',
        'multiplier': 100,
    },
}

# Draw EL and VASS in separate panels so the alternative area systems are not
mixed.
for column, settings in LINE_PLOT_SETTINGS.items():
    fig, axes = plt.subplots(1, 2, figsize=(16, 6), sharey=True)

    for ax, area_type, panel_title in [
        (axes[0], 'EL', 'Electricity price areas (EL)'),
        (axes[1], 'VASS', 'Watercourse areas (VASS)'),
    ]:
        area_system_data = plot_data.loc[plot_data['area_type'] == area_type]

        for area_label, area_data in area_system_data.groupby('area_label',
sort=True):
            ax.plot(
                area_data['date'],
                area_data[column] * settings['multiplier'],
                label=area_label,
                linewidth=1.5,
            )

    # Draw the national series last, so it remains visible above the area
lines.
    ax.plot(
        national_data['date'],
        national_data[column] * settings['multiplier'],
        color='black',
        label='NO-0',
        linewidth=2.2,
        linestyle='--',
    )
    ax.set_title(panel_title)
    ax.set_xlabel('Date')
    ax.set_ylabel(settings['y_label'])
    ax.grid(alpha=0.3)
    ax.legend(
        title='Area',
        loc='upper center',
        bbox_to_anchor=(0.5, -0.17),
        ncol=3,
        frameon=False,
    )

    fig.suptitle(settings['title'], y=0.98)
    fig.autofmt_xdate()
    fig.tight_layout()
    fig.subplots_adjust(bottom=0.25)
    plt.show()

# Capacity is a size measure, so compare areas with bars instead of a date
axis.
latest_date = plot_data['date'].max()
latest_capacity = plot_data.loc[plot_data['date'] == latest_date].copy()
national_capacity = latest_capacity.loc[
    (latest_capacity['area_type'] == 'NO')
    & (latest_capacity['area_number'] == 0)

```

```

]

fig, axes = plt.subplots(1, 2, figsize=(16, 6), sharey=True)

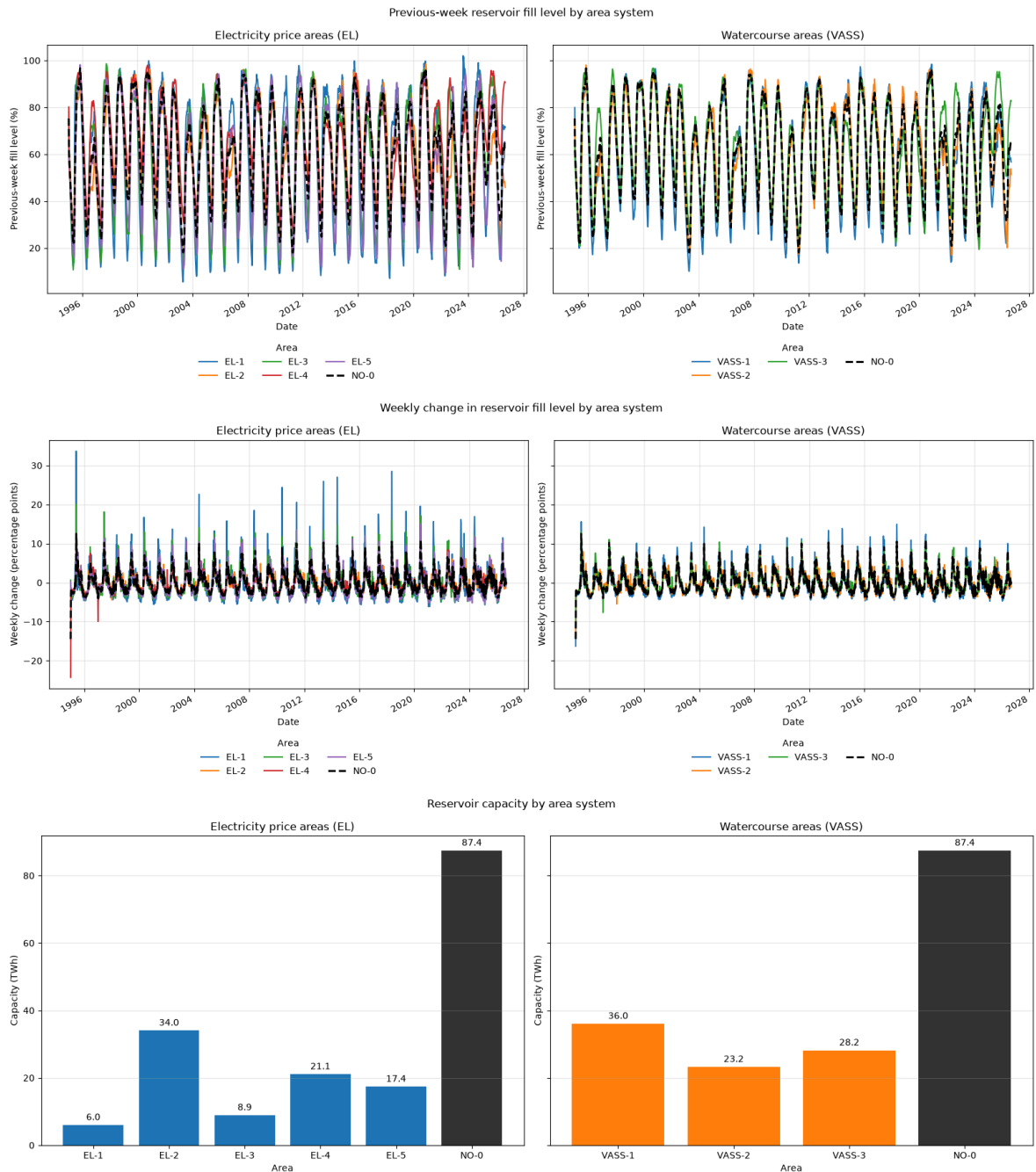
for ax, area_type, panel_title, area_color in [
    (axes[0], 'EL', 'Electricity price areas (EL)', '#1f77b4'),
    (axes[1], 'VASS', 'Watercourse areas (VASS)', '#ff7f0e'),
]:
    area_capacity = latest_capacity.loc[
        latest_capacity['area_type'] == area_type
    ].sort_values('area_number')
    comparison_capacity = pd.concat([area_capacity, national_capacity])
    bar_colors = [area_color] * len(area_capacity) + ['#333333']

    bars = ax.bar(
        comparison_capacity['area_label'],
        comparison_capacity['capacity_twh'],
        color=bar_colors,
    )
    ax.bar_label(bars, fmt='%.1f', padding=3)
    ax.set_title(panel_title)
    ax.set_xlabel('Area')
    ax.set_ylabel('Capacity (TWh)')
    ax.grid(axis='y', alpha=0.3)

fig.suptitle('Reservoir capacity by area system', y=0.98)
fig.tight_layout()
plt.show()

```





6. Combined multi-axis overview for Norway

The national measurement columns have different units and some are mathematically related. This single figure therefore keeps the original units instead of normalizing them. The top panel uses a percentage axis for fill levels and a TWh axis for stored energy and capacity. The lower panel shows weekly change in percentage points. Together, the two panels provide one readable overview of all five measurement columns.

```
In [6]: import sys

# Make the reusable project module available when Jupyter starts in notebooks/.
PROJECT_CODE_PATH = PROJECT_ROOT / 'Compulsory Assignments'
if str(PROJECT_CODE_PATH) not in sys.path:
    sys.path.insert(0, str(PROJECT_CODE_PATH))

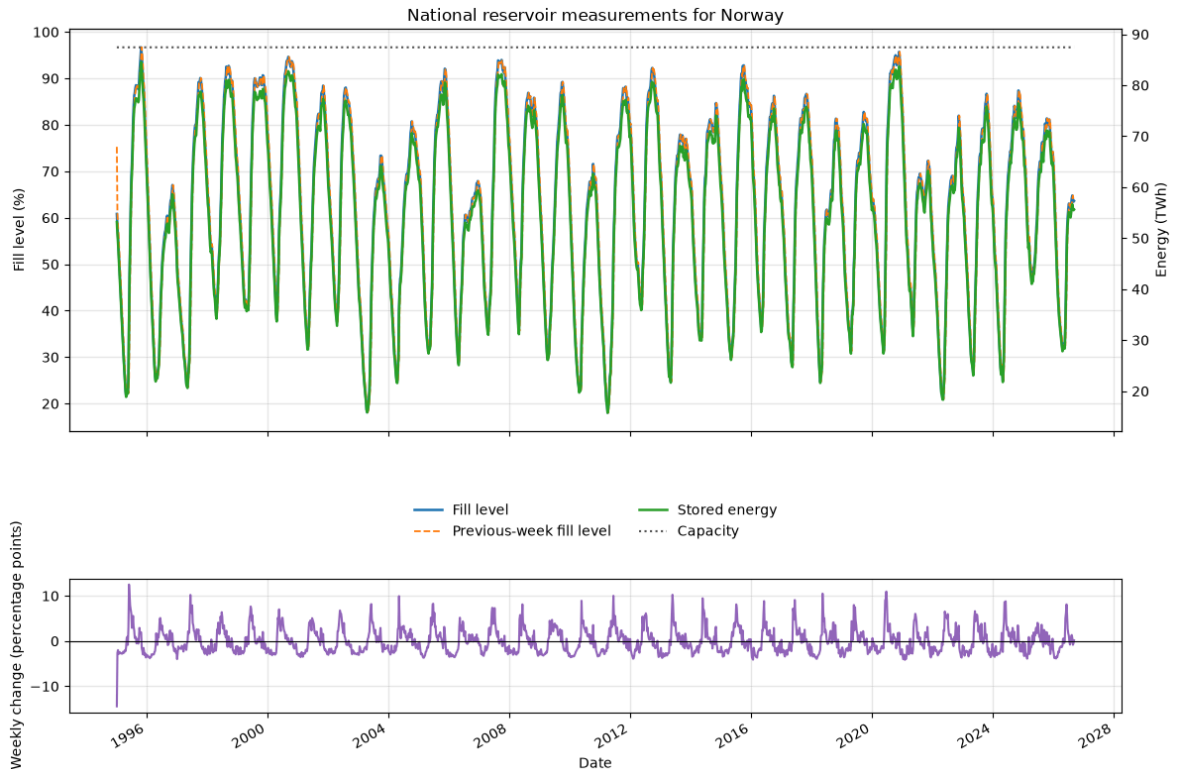
from modules.plotting import plot_national_overview
```

```

# Select the single national time series so the combined figure remains
readable.
national_data = df.loc[
    (df['area_type'] == 'NO') & (df['area_number'] == 0)
].copy()

# Use the reusable function so the full and short time-period charts share
style.
plot_national_overview(
    national_data,
    'National reservoir measurements for Norway',
)

```



7. Detailed overview of the last five years

This version uses the same variables and units as the full-period overview, but only includes the five years leading up to the most recent observation. Fewer observations make recent changes easier to inspect.

```

In [7]: # Find the date five years before the latest available observation.
five_year_start = national_data['date'].max() - pd.DateOffset(years=5)
recent_data = national_data.loc[national_data['date'] >=
    five_year_start].copy()

print(
    f'Detailed period: {recent_data["date"].min().date()} '
    f'to {recent_data["date"].max().date()}'
)
print(f'Observations in detailed period: {len(recent_data)}')

# Reuse the exact same plot design with only the shorter data period changed.
plot_national_overview(
    recent_data,

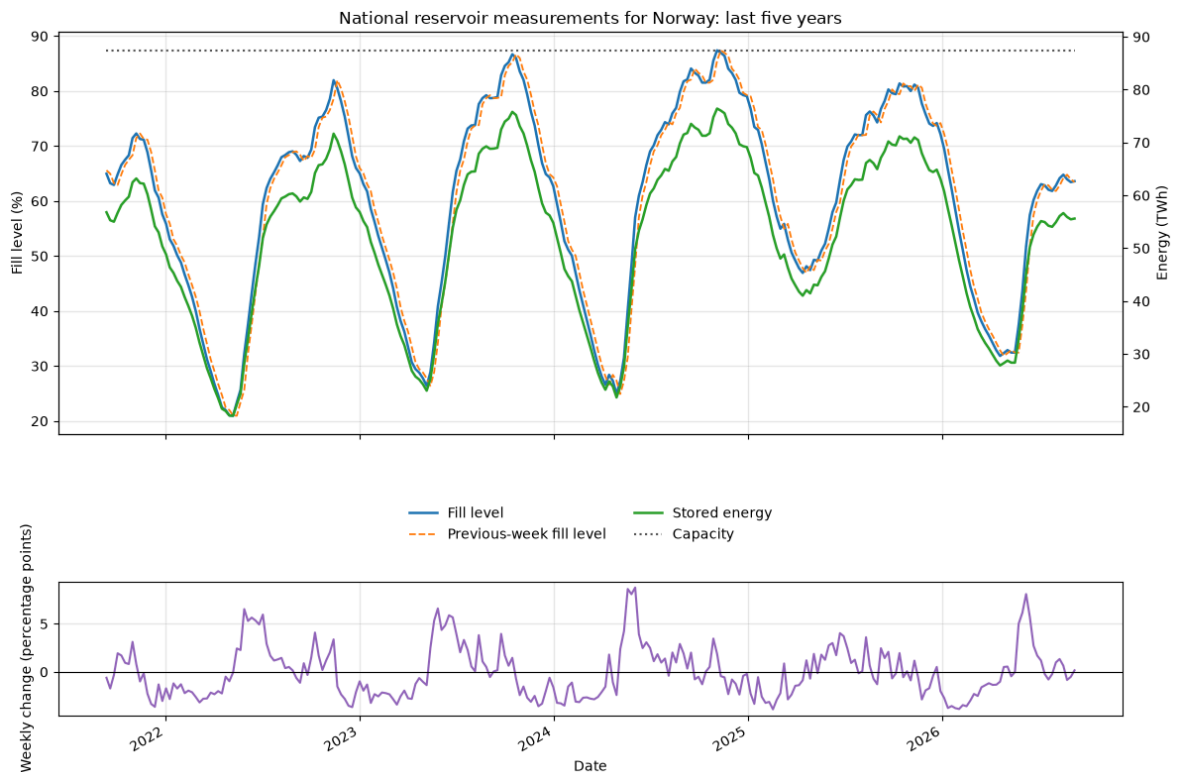
```

'National reservoir measurements for Norway: last five years',

)

Detailed period: 2021-09-12 to 2026-09-06

Observations in detailed period: 261



8. Project links

- Public GitHub repository: <https://github.com/EliasVinsrygg/IND320-EliasVinsrygg>
- Published Streamlit app: <https://ind320-eliasvinsrygg.streamlit.app/>

9. Compulsory Assignment 1 work log

This is a working draft. Short reflections from later exercises will be added

Notebook: The compulsory work began in Jupyter Notebook, where I explored the weekly Norwegian reservoir dataset. After a while, I understood the relation between EL, VASS and the overall NO-0. I created separate visualisations for EL and VASS areas because the two regional systems are independent from each other. I also made a recent-years view to make long time series easier to interpret, because the full-time plots are kind of noisy.

Streamlit: The app is organised into four pages. I learned how to separate data loading from page code and cache the local CSV file so it is not read repeatedly. The interactive chart uses Plotly, but I tried with Altair first. A main design decision was to show Norway (NO-0) as a clear national reference while allowing either EL or VASS regions to be displayed, if the user wants to. The graph shows the first month in the dataset, because it was a task requirement. However, I think it makes more sense to show recent data. I therefore added the "Show recent years" button.

A useful Streamlit lesson was that interaction state needs deliberate handling. The selected time period must remain unchanged when a regional view is selected, so the app keeps the chart period separately from the slider's temporary UI value

10. AI usage

I use ChatGPT and Codex actively as development tools when working on the assignment. My goal is to learn to be an efficient developer by using agentic coding to improve productivity without compromising my understanding or the quality of the final product.

My typical workflow is to discuss problems and possible solutions with the AI agent before arriving at a solution that I review and approve. Codex is then used to generate or modify code based on specific instructions. Code that I've written myself is also looked over by Codex, so I can get input on how to improve it. By using Codex directly in the terminal, I can also ask it to push my changes to git or change my Github repository name if I want to do that quickly. But that's also stuff I do on my own normally.

An important principle for me is that I should understand the code and the solutions we develop. I frequently ask the agent to explain problems and suggest some alternatives.

11. Screencast

The public link to the five-minute screencast will be added here before the Canvas submission.